主流开源 LLM

主流开源 LLM 模块级技术速记表（面试参考）

> 覆盖 DeepSeek V3/R1、DeepSeek V3.2、Qwen3、Qwen3.5、Llama 3/3.1、Llama 4 更新日期：2026-03-15

一、模块级技术速记表

模块	DeepSeek V3 / R1	DeepSeek V3.2	Qwen3	Qwen3.5	Llama 3 / 3.1	Llama 4
发布时间	2024.12 / 2025.01	2025.12	2025.04	2026.02	2024.04 / 2024.07	2025.
总参数	671B	685B	0.6B~235B	0.8B~397B	8B~405B	109B~2T+
激活参数	37B	37B	全部(Dense) / 3B~22B(MoE)	17B (MoE)	全部 (Dense)	17B (MoE)
Dense / MoE	MoE	MoE	Dense + MoE	MoE	Dense	MoE
注意力机制	**MLA** (Multi-head Latent Attention)	**MLA + DSA** (DeepSeek Sparse Attention)	**GQA** + QK-Norm	**Gated DeltaNet** (线性注意力) + Gated Attention (3:1混合)	**GQA**	**GQA**
位置编码	RoPE	RoPE	RoPE + YARN + DCA	RoPE + YARN	RoPE	**iRoPE** (交错式 RoPE)
FFN 激活函数	SwiGLU	SwiGLU	SwiGLU	SwiGLU	SwiGLU	SwiGLU
归一化	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pr norm)
MoE 专家数/激活数	256 路由 + 1 共享 / Top-8	同 V3	128 / Top-8（无共享）	有共享专家 + 路由专家	— (Dense)	16(Scout) / 128(Maverick)
MoE 路由函数	Sigmoid + 无辅助损失 (bias调节)	同 V3	Global-batch 负载均衡损失	共享专家 + learned gating	—	Learned gatin
训练精度	**FP8** 混合	**FP8** 混合	BF16	BF16	BF16	**FP8**
训练数据量	14.8T token	继续训练 ~1T	36T token	万亿级多模态 token	15T+ token	30T+ token
上下文长度	128K	128K	32K→128K (YARN)	**262K** (原生)→1M (YARN)	128K	1M~**10M**
RL 算法	**GRPO** (无Critic)	改进 GRPO (DAPO/ Dr.GRPO)	大规模 RL	百万级 Agent 环境 RL	**PPO**	SFT + RL
多模态	无（纯文本）	无（纯文本）	无（纯文本 LLM）	**Early Fusion** (原生视觉语言)	适配器方式(3.2)	**Early Fusion**
视觉编码器	—	—	—	DeepStack ViT (Conv3d)	ViT-H/14 + Cross-Attn Adapter	统一骨干
特色训练技术	MTP (多token预测)	DSA 稀疏注意力	Thinking/Non-Thinking 混合	Gated DeltaNet 线性注意力	4D 并行 (16K GPU)	MetaP 超参数 优 + 蒸馏
Tokenizer	BPE ~128K	同 V3	BBPE 151,669	~250K	BPE 128K	BPE ~128K
许可证	MIT	MIT	Apache 2.0	Apache 2.0	Llama License	Llama License

---

二、注意力机制速记

技术	使用者	核心思路	复杂度
MHA (Multi-Head Attention)	经典Transformer	每个head独立的Q/K/V	$O(L^2 \cdot d)$
GQA (Grouped Query Attention)	Qwen3, Llama 3/4	多个Q head共享一组K/V head	$O(L^2 \cdot d)$，KV cache减少
MLA (Multi-head Latent Attention)	DeepSeek V3/R1/V3.2	K/V压缩为低秩隐向量，推理时KV cache极小	$O(L^2 \cdot d)$，KV cache大幅减少
DSA (DeepSeek Sparse Attention)	DeepSeek V3.2	indexer选Top-k个token做注意力	$\mathbf{O(L \cdot k)}$，$k \ll L$
Gated DeltaNet (线性注意力)	Qwen3.5	Mamba2门控衰减 + Delta Rule更新隐状态	$\mathbf{O(L \cdot d)}$，线性复杂度
iRoPE (Interleaved RoPE)	Llama 4	交错使用RoPE层和无位置编码层	$O(L^2 \cdot d)$，但支持超长上下文

三、MoE 路由策略速记

技术	使用者	关键点
Sigmoid路由 + 无辅助损失	DeepSeek V3/R1/V3.2	用bias手动调节负载，取消辅助损失避免干扰亲和度
Global-batch 负载均衡损失	Qwen3	全局batch级别的辅助损失，鼓励专家特化；取消共享专家
共享专家 + 路由专家	Qwen2.5-MoE, Qwen3.5	共享专家处理所有token的通用特征，路由专家做专业化
交替Dense/MoE层	Llama 4	每隔一个block交替使用Dense FFN和MoE

四、RL 训练策略速记

技术	使用者	关键点
GRPO (Group Relative Policy Optimization)	DeepSeek R1/V3.2	去掉Critic网络，从同prompt的一组(16个)回答中算相对优势；规则奖励(无神经RM)防reward hacking
PPO (Proximal Policy Optimization)	Llama 3	需要Policy+Critic两个网络，计算量大但稳定
大规模Agent RL	Qwen3.5	百万级Agent环境 + 异步RL框架，面向长期多步骤agentic任务
Thinking/Non-Thinking RL	Qwen3	通过RL训练出单模型的思考/直答双模式 + thinking budget机制
改进GRPO (DAPO + Dr.GRPO)	DeepSeek V3.2	非对称裁剪 + 动态采样 + token级损失 + 过滤零梯度样本

五、Qwen3.5 关键新增信息（2026.02）

Qwen3.5 于 2026 年 2 月 16 日发布，旗舰模型为 Qwen3.5-397B-A17B（397B 总参数，17B 激活参数）。

架构核心变化（相对 Qwen3）

（1）Gated Delta Networks（线性注意力）

- 采用 3:1 混合注意力布局：每 3 个 Transformer block 使用 Gated DeltaNet（线性注意力），每第 4 个 block 使用 Gated Attention（全注意力/softmax）
- Gated DeltaNet 结合了 Mamba2 的门控衰减机制与 Delta Rule 的隐状态更新方式
- 实现了相对序列长度的线性复杂度，使模型能够在极大上下文下保持性能
- 注意力输出门控机制有助于消除 Attention Sink 和 Massive Activation 问题，确保大规模训练数值稳定
- 相比 Qwen3 的纯 GQA 方案，这是一次根本性的注意力架构革新

（2）MoE 恢复共享专家

- Qwen3.5 重新引入共享专家机制：一个专用的Dense MLP处理所有token以捕获通用特征，提升训练稳定性
- 不同于 Qwen3 取消共享专家的策略，Qwen3.5 又加回来了

（3）原生多模态

- Qwen3.5 不再是独立的文本模型加视觉适配器，而是统一的视觉语言骨干，从预训练起就融合文本和多模态token
- 视觉编码器采用 DeepStack ViT，使用 Conv3d 进行 patch embedding 以原生捕捉时间动态；融合多层特征而非仅用最后一层

（4）上下文与词表

- 原生支持 262,144 token 上下文，可通过 YARN 扩展至约 1M token
- 词表扩展到约 250K，对中文字符、数学符号和代码token编码更紧凑

（5）多语言

- 支持 201 种语言和方言（Qwen3 为 119 种）

训练流程

- 在万亿级多模态token上进行 Early Fusion 训练，多模态训练效率接近纯文本训练的100%
- RL 训练扩展到百万级Agent环境，任务分布逐步增加复杂度，采用异步RL框架
- 解码吞吐量在32K上下文下比Qwen3-Max快8.6倍，256K上下文下快19倍

模型规格一览

模型	类型	总参数	激活参数	上下文
Qwen3.5-397B-A17B	MoE	397B	17B	262K
Qwen3.5-122B-A10B	MoE	122B	10B	262K
Qwen3.5-35B-A3B	MoE	35B	3B	262K
Qwen3.5-27B	Dense	27B	27B	262K
Qwen3.5-9B	Dense	9B	9B	262K
Qwen3.5-4B	Dense	4B	4B	262K
Qwen3.5-0.8B	Dense	0.8B	0.8B	262K

六、面试一句话速记

考点	一句话回答
DeepSeek MLA 是什么?	将 KV 压缩为低秩隐向量，比 GQA 更激进地减少 KV cache，同时不损失甚至提升性能
DeepSeek DSA 是什么?	V3.2 新增的稀疏注意力，用 indexer 选 Top-k 个 token 做注意力，复杂度从 $O(L^2)$ 降到 $O(L \cdot k)$
GRPO 和 PPO 的区别?	GRPO 去掉 Critic 网络，从同一 prompt 的一组回答中计算相对优势（均值/标准差归一化），计算量减半
DeepSeek 无辅助损失路由?	取消辅助损失，改用 bias 项手动调节专家负载，负载均衡略差但模型质量更好
Qwen3 Thinking 模式?	单模型通过 chat template 切换思考/直答模式，支持 thinking budget 控制推理时思考量
Qwen3.5 Gated DeltaNet?	3:1 混合布局（3层线性注意力 + 1层全注意力），线性复杂度，支持超长上下文
Qwen3.5 相比 Qwen3 变了什么?	Gated DeltaNet 替代纯GQA；恢复共享专家；原生多模态Early Fusion；上下文262K；201种语言；250K词表
Llama 4 iRoPE?	交错使用 RoPE 层和无位置编码层，突破传统位置编码限制，支持最长 10M token
Llama 4 vs Llama 3 最大变化?	从 Dense 变为 MoE；从后融合变为 Early Fusion 多模态；上下文从 128K 到 10M；BF16 变 FP8
为什么用 FP8 训练?	见下方「FP8 训练详解」
哪些模型用了 MTP?	DeepSeek V3（预测2个额外token，顺序预测保持因果链）；Qwen3.5 也支持 MTP speculative decoding

---

七、FP8 训练专题详解（高频面试考点）

1. 为什么要用 FP8?

传统 LLM 训练使用 BF16-FP32 混合精度。FP8 是从 16-bit 到 8-bit 的自然演进：

收益维度	具体效果
计算速度	H100/H200 Tensor Core 的 FP8 FLOPS 是 BF16 的 **2 倍**，实测端到端加速 **50~75%**
显存节省	权重/激活/梯度存储减半，单卡可训练更大模型（同集群下可训约 2 倍参数的模型）
通信带宽	分布式训练中梯度通信量减半，减少跨节点通信开销
成本	DeepSeek V3 全量训练仅 278.8 万 H800 GPU 小时（约 557 万美元），FP8 是关键因素之一

2. FP8 的两种格式

格式	指数位 / 尾数位	最大值	用途	特点
E4M3	4月3日	±448	前向传播（权重、激活）	精度更高（尾数多），适合需要精细数值的前向计算
E5M2	5月2日	±57344	反向传播（梯度）	动态范围更大（指数多），适合梯度的大范围波动

混合策略：前向用 E4M3（精度优先），反向用 E5M2（范围优先）。这是 NVIDIA Transformer Engine 的默认配置。

> DeepSeek V3 的创新：前向和反向均使用 **E4M3**，通过细粒度量化（block-wise scaling）来弥补 E4M3 范围不足的问题，反而获得更好的精度。

3. FP8 训练的核心挑战

（1）动态范围窄 → 溢出/下溢

- E4M3 最大值仅 448，远小于 BF16 的 ~3.4×10^{38}
- 激活值中的离群值（outlier）可能远超 FP8 可表示范围，导致溢出
- 梯度值可能很小，低于 FP8 最小精度，导致下溢（梯度消失）

（2）量化误差累积

- 8-bit 精度低，每次运算引入的舍入误差更大
- 在数百层 Transformer 中逐层传播，可能导致训练不稳定甚至 loss 爆炸

（3）Attention Sink / Massive Activation

- Transformer 残差流中容易产生极端离群值（幅度可达标准差的 1000 倍以上）
- 这些离群值在 FP8 的窄范围内无法精确表示，是 FP8 训练不稳定的主因

4. 解决方案

技术	说明	使用者
Per-tensor Scaling	每个张量维护一个缩放因子，将数值映射到 FP8 范围内	早期方案（粗粒度，易受离群值影响）
Block-wise / Tile-wise Scaling	将张量分成小块，每块一个独立缩放因子，离群值不会污染全局精度	**DeepSeek V3**（关键创新）、Ling 2.0
Delayed Scaling	基于历史 amax 统计来预估缩放因子，避免每步重新计算	NVIDIA Transformer Engine 默认方案
精度解耦	不同组件用不同精度：权重/激活用 FP8，master weights/optimizer states 保持 FP32	通用最佳实践
前向 E4M3 + 反向 E5M2	前向需要精度，反向需要范围，各取所需	标准混合方案（NVIDIA 推荐）
前向反向均 E4M3 + fine-grained scaling	DeepSeek 的创新：用细粒度缩放弥补 E4M3 范围不足，获得更好精度	DeepSeek V3/V3.2
TWEO 正则化	对残差块输出加正则项，抑制离群值产生，从源头解决问题	学术前沿（2025.11）

5. DeepSeek V3 的 FP8 实现要点（面试重点）

DeepSeek V3 是首个在超大规模（671B）模型上验证 FP8 训练可行性的工作：

- **量化粒度**：采用 **block-wise scaling**（细粒度量化），权重用 per-block 量化，激活用 per-token 量化
- **前向+反向均用 E4M3**：不用传统的 E4M3+E5M2 混合，而是全程 E4M3 + 细粒度缩放
- **算法-框架-硬件协同设计**：
  - 解决跨节点 MoE 训练的通信瓶颈
  - 实现计算与通信的近完全重叠
  - FP8 通信进一步减少分布式训练带宽需求
- **效果**：与 BF16 训练结果几乎无损，loss 相对误差约 0.001

6. 面试一句话速记

问题	回答
FP8 为什么快?	H100 Tensor Core FP8 算力是 BF16 的 2 倍，内存减半，通信减半
E4M3 和 E5M2 区别?	E4M3 精度高（3位尾数）用于前向，E5M2 范围大（5位指数）用于反向梯度
FP8 最大挑战?	动态范围窄（E4M3 最大仅448），Transformer 中的激活离群值容易溢出导致训练崩溃
怎么解决离群值?	核心是 Scaling 策略：从 per-tensor → block-wise fine-grained scaling，局部缩放避免离群值污染全局
DeepSeek V3 FP8 有什么创新?	①前向反向均用 E4M3 + block-wise scaling（不用 E5M2）；②首次在 671B MoE 上验证可行；③FP8 通信减少分布式开销
哪些部分不能用 FP8?	Master weights、optimizer states（一阶/二阶动量）通常保持 FP32，LayerNorm/RMSNorm 的累加也常用高精度

主流开源 LLM 模块级技术速记表（面试参考）

覆盖 DeepSeek V3/R1/V3.2、Qwen3/Qwen3.5、Llama 3/3.1/4

更新日期：2026-03-25

一、模块级技术速记总表

模块	DeepSeek V3/R1	DeepSeek V3.2	Qwen3	Qwen3.5	Llama 3/3.1	Llama 4
发布时间	2024.12 / 2025.01	2025.12	2025.04	2026.02	2024.04 / 2024.07	2025.
总参数	671B	685B	0.6B~235B	0.8B~397B	8B~405B	109B~2T+
激活参数	37B	37B	全部 (Dense) / 3B~22B(MoE)	17B (MoE)	全部 (Dense)	17B (MoE)
Dense / MoE	MoE	MoE	Dense + MoE	MoE	Dense	MoE
					GQA	GQA

注意力机制	MLA (Multi-head Latent Attention)	MLA + DSA (DeepSeek Sparse Attention)	GQA + QK-Norm	Gated DeltaNet（线性注意力）+ Gated Attention（3:1 混合）		
位置编码	RoPE	RoPE	RoPE + YARN + DCA	RoPE + YARN	RoPE	iRoPE（交错式 RoPE）
FFN 激活函数	SwiGLU	SwiGLU	SwiGLU	SwiGLU	SwiGLU	SwiGLU
归一化	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)	RMSNorm (pre-norm)
MoE 专家数/激活数	256 路由 + 1 共享 / Top-8	同 V3	128 / Top-8（无共享）	有共享专家 + 路由专家	—（Dense）	16(Scout) / 128(Maverick)
MoE 路由函数	Sigmoid + 无辅助损失（bias调节）	同 V3	Global-batch 负载均衡损失	共享专家 + learned gating	—	Learned gating
训练精度	FP8 混合	FP8 混合	BF16	BF16	BF16	FP8
训练数据量	14.8T token	继续训练 ~1T	36T token	万亿级多模态 token	15T+ token	30T+ token
上下文长度	128K	128K	32K→128K (YARN)	**262K**（原生）→1M (YARN)	128K	1M~**10M**
RL 算法	GRPO（无 Critic）	改进 GRPO (DAPO / Dr.GRPO)	大规模 RL	百万级 Agent 环境 RL	PPO	SFT + RL
多模态	无（纯文本）	无（纯文本）	无（纯文本 LLM）	**Early Fusion**（原生视觉语言）	适配器方式 (3.2)	Early Fusion
视觉编码器	—	—	—	DeepStack ViT (Conv3d)	ViT-H/14 + Cross-Attn Adapter	统一骨干

特色训练技术	MTP（多token预测）	DSA 稀疏注意力	Thinking/Non-Thinking 混合	Gated DeltaNet 线性注意力	4D 并行（16K GPU）	MetaP 超参数优 + 蒸馏
Tokenizer	BPE ~128K	同 V3	BBPE 151,669	~250K	BPE 128K	BPE ~128K
许可证	MIT	MIT	Apache 2.0	Apache 2.0	Llama License	Llama License

---

二、注意力机制完整速记

2.1 六大变体核心对比

技术	使用者	KV 头数	复杂度	显存	核心思路
MHA 多头注意力	经典 Transformer、GPT-2、BERT	= Q 头数	$O(L^2 \cdot d)$	最高	每个 head 独立的 Q/K/V，表达力最强
GQA 分组查询注意力	Qwen3、Llama 3/4	= G 组	$O(L^2 \cdot d)$，KV cache 减少	中等	多个 Q head 共享一组 KV head，G 通常为 4/8
MLA 多头潜在注意力	DeepSeek V2/V3/R1/V3.2	低秩压缩	$O(L^2 \cdot d)$，KV cache 大幅减少	极低	K/V 压缩为低秩隐向量，推理时 KV cache 极小
DSA 稀疏注意力	DeepSeek V3.2	—	$O(L \cdot k)$，$k \ll L$	低	indexer 选 Top-k 个 token 做注意力
Gated DeltaNet 线性注意力	Qwen3.5	—	$O(L \cdot d)$，线性复杂度	极低	Mamba2 门控衰减 + Delta Rule 更新隐状态
	Llama 4	= G 组		低	

iRoPE 交错式 RoPE			$O(L^2 \cdot d)$，但支持超长上下文		交错使用 RoPE 层和无位置编码层

2.2 MLA 深度拆解（DeepSeek 核心创新）

> 💡 核心公式思路：KV = W_KV · c，其中 c 是低维压缩向量（如 512→128 维），推理时只缓存 c，解压在飞。KV cache 显存降低约 5–8×。

- **训练时**：Q 也做低秩分解（c_Q = W_Q' · x），再升维得多头 Q
- **推理时**：将 W_KV 吸收进 W_Q，消除解压开销，速度接近 MQA
- **RoPE 处理**：位置编码只加在「解耦」出来的 Q_rope / K_rope 上，不影响主 KV 压缩
- **对比 GQA**：GQA 降低头数，MLA 降低每头维度，方向不同，效果互补

2.3 Qwen3.5 Gated DeltaNet 线性注意力拆解

> Qwen3.5 相比 Qwen3 纯 GQA 方案，这是一次根本性的注意力架构革新。

- **3:1 混合布局**：每 3 个 Transformer block 使用 Gated DeltaNet（线性注意力），每第 4 个 block 使用 Gated Attention（全注意力/softmax）
- **Gated DeltaNet 机制**：结合了 Mamba2 的门控衰减机制与 Delta Rule 的隐状态更新方式
- **线性复杂度**：实现了相对序列长度的线性复杂度 $O(L \cdot d)$，使模型能够在极大上下文下保持性能
- **消除 Attention Sink**：注意力输出门控机制有助于消除 Attention Sink 和 Massive Activation 问题，确保大规模训练数值稳定

2.4 位置编码速记

方案	绝对/相对	外推能力	代表模型	关键点
Sinusoidal	绝对	差	原版 Transformer	固定不学习，超出训练长度性能下降

ALiBi	相对	好	早期 OPT 变种	在 attention score 加线性偏置，不修改 embedding
RoPE	相对	中（需扩展）	Llama、Qwen、DeepSeek	旋转矩阵作用于 QK，内积自然编码相对位置，主流选择
YaRN	相对	极好	Qwen3.5、DeepSeek	对 RoPE 频率重缩放，低频保持原始，高频压缩
NTK-aware	相对	好	Llama 长上下文版	对 RoPE base 做 NTK 缩放，性价比高
iRoPE	相对	极好	Llama 4	交错使用 RoPE 层和无位置编码层，支持最长 10M token
DCA	相对	好	Qwen3	Dual Chunk Attention，分块处理长序列

2.5 Attention Sink / Massive Activation 问题

> ⚠ 残差流中少数维度出现幅值极端的激活值（达 σ 的 1000×），FP8/量化中难以表示，导致训练不稳定。首个 token（[BOS]）往往成为"sink token"，承接大量无意义注意力权重。

- **Qwen3.5 Gated DeltaNet**：输出门控机制直接消除 Attention Sink，使 FP8 训练在超长序列下保持数值稳定
- **DeepSeek V3**：细粒度 block-wise 量化，不裁剪离群值，用更细的缩放因子包容离群值
- **LLaMA 系列**：保留 BOS token 允许模型"泄压"，绕开而非解决问题

三、MoE 路由策略完整速记

3.1 路由机制全景对比

路由策略	使用者	选专家数 K	核心机制	负载均衡	关键点
Top-K Token-choice	GShard、Switch	K=1 或 2	每 token 用 softmax 打分选前 K 专家；得分作为加权系数	差（需辅助损失）	最基础方案，马太效应明显
Expert-choice	Google ST-MoE	固定容量 C	每专家主动选前 C 个 token；天然均衡，但 token 可能被丢弃	好	自回归推理难以实现
Global-batch 负载均衡损失	Qwen3	K=8（无共享）	全局 batch 级别的辅助损失，鼓励专家特化；取消共享专家	中等	取消共享专家，全靠路由学习
Sigmoid 路由 + 无辅助损失	DeepSeek V3/R1/V3.2	K=8（细粒度）	每专家维护可更新偏置 b_i，动态调整代替辅助损失，避免干扰亲和度	优	零污染主 loss
共享专家 + 路由专家	Qwen2.5-MoE、Qwen3.5、DeepSeek V2	K_s 共享 + K_r 路由	共享专家处理所有 token 的通用特征，路由专家做专业化	中	提升训练稳定性
交替 Dense/MoE 层	Llama 4	—	每隔一个 block 交替使用 Dense FFN 和 MoE	—	Scout: 16专家选1；Maverick: 128专家

3.2 DeepSeek V3 路由策略深度拆解

> ✅ 核心创新：细粒度 MoE + 无辅助损失负载均衡

- **细粒度专家（Fine-grained MoE）**：将每个专家 FFN 拆为更小单元（如 256 个小专家），选更多个（K=8），等效计算量不变但路由更精细，表达力更强
- **无辅助损失均衡**：每专家维护偏置项 `b_i`；路由分数 = 原始 score + b_i；过载专家 b_i 降低，欠载专家 b_i 升高；偏置仅影响路由决策，**不计入加权求和，主 loss 零污染**
- **Sigmoid 路由函数**：替代 softmax，允许各专家分数独立，不归一化，避免不必要的竞争
- **序列级均衡约束（Sequence-level balance）**：在一个序列内对专家负载做软约束，训练更稳
- **Token dropping**：超出专家容量时丢弃部分 token，推理时关闭 dropping 保证完整性
- **通信优化**：671B 模型跨节点 MoE 通信用 FP8 量化后传输，带宽减半；计算与通信几乎完全 overlap

3.3 负载均衡方法详细对比

方法	原理	优点	缺点	典型参数
辅助损失（Auxiliary loss）	L_total = L_main + α × L_balance；L_balance 惩罚专家负载方差	实现简单	污染主 loss 梯度，α 需调	α ≈ 0.01–0.1
动态偏置（DeepSeek V3）	每步更新 b_i：过载则 b_i -= γ，欠载则 b_i += γ	不污染主 loss，均衡效果好	引入额外状态，实现稍复杂	γ ≈ 0.001
Expert-choice routing	专家主动选 token（反向选择）	天然完美均衡	自回归推理困难，token 可能被丢弃	每专家容量 C = 2× 平均负载
随机路由	以 score 为概率随机选专家	简单	方差大，训练不稳定	—

3.4 共享专家机制演化

模型	共享专家	设计意图
DeepSeek V2	有（2 个）	捕获通用特征，提升训练稳定性

Qwen3-MoE	**无**	减少参数冗余，全靠路由专家学习
Qwen3.5	**有（1 个 Dense MLP）**	重新引入，发现对多模态训练稳定性有帮助
DeepSeek V3/R1	有（1 个）	处理所有 token 的基础特征，路由专家处理特定信息
Llama 4	无	依赖交替 Dense/MoE 层设计

3.5 MoE 推理优化要点

> 💡 面试高频：解释 MoE 推理为什么比 Dense 省计算但通信代价大

- **参数稀疏激活**：671B 参数模型每次前向只激活约 37B（K/N × 全量），FLOPS 与小模型相当
- **All-to-All 通信**：Token 被路由到不同节点的专家，需跨节点传输，是 MoE 推理延迟主要瓶颈
- **Expert Parallelism（EP）**：将专家分布到多卡/多节点，每卡只存部分专家；需要 token dispatch 和 gather
- **批处理友好性**：大 batch 时专家负载更均匀，推理效率高；小 batch 时专家利用率低，是 MoE 短板
- **Speculative Decoding + MoE**：draft model 用小 Dense，验证用 MoE，能有效提升生成速度

四、RL 训练策略速记

4.1 主流 RL 算法对比

技术	使用者	核心机制	优点	缺点	关键点
GRPO（Group Relative Policy	DeepSeek R1/V3.2	去掉 Critic 网络，从同一 prompt 的一组	无需 Critic，显	需多次采样，速度较	规则奖励（无神经网络 RM）防 reward

Optimization）		（16个）回答中计算相对优势（均值/标准差归一化）	存减半；适配可验证奖励	慢	hacking
PPO（Proximal Policy Optimization）	Llama 3	Clipped surrogate objective；需要 Policy + Critic 两个网络	稳定，样本效率高	需要 4 个模型，计算量大但稳定	InstructGPT 经典方案
大规模 Agent RL	Qwen3.5	百万级 Agent 环境 + 异步 RL 框架，面向长期多步骤 agentic 任务	支持复杂任务	环境构造成本高	任务分布逐步增加复杂度
Thinking/Non-Thinking RL	Qwen3	通过 RL 训练出单模型的思考/直答双模式 + thinking budget 机制	一个模型支持两种模式	训练复杂	支持 thinking budget 控制推理时思考量
改进 GRPO（DAPO + Dr.GRPO）	DeepSeek V3.2	非对称裁剪 + 动态采样 + token 级损失 + 过滤零梯度样本	比原始 GRPO 更稳定高效	实现更复杂	解决原始 GRPO 训练不稳定问题
DPO	Llama 3 对齐阶段	将 RL 改写为分类目标，直接在偏好数据上优化	无需 RM，训练稳定	需高质量偏好对，泛化弱	适合对齐微调阶段
REINFORCE++	轻量 RL 场景	经典 REINFORCE + KL 惩罚 + 优势归一化	无 Critic，实现最简单	方差大，需大量样本	—

4.2 GRPO 深度拆解（DeepSeek-R1 核心）

> 公式：A_i = (r_i − mean(r_{1..G})) / std(r_{1..G})，每个输出的优势 = 组内相对排名

- **无 Critic 模型**：用 G 个采样输出的奖励均值作为 baseline，省去 Critic 网络（节约约 25% 显存）
- **KL 散度惩罚**：L = -A_i · log π_θ(a|s) + β · KL(π_θ || π_ref)，防止模型偏离参考策略太远
- **可验证奖励（Verifiable Reward）**：数学题对/错 +1/-1，代码能否通过测试，不需要人工标注的奖励模型
- **格式奖励**：奖励按规定格式（如 `<think>...</think>`）输出，引导 CoT 出现
- **冷启动**：DeepSeek-R1 先用少量长思维链数据 SFT，再做 GRPO，避免纯 RL 的不稳定性

4.3 RL 训练稳定性技巧

问题	现象	解决方案
奖励黑客（Reward hacking）	模型找到奖励函数漏洞，输出冗长无意义内容	KL 惩罚、奖励裁剪（clip reward）、设置长度惩罚
模式崩溃	模型只输出少数几种高奖励模式	增大采样温度、多样性奖励、提高 G（组大小）
训练不稳定	Loss 震荡或爆炸	优势归一化、梯度裁剪、学习率 warmup
分布偏移	策略与参考模型差距过大	β-KL 惩罚、定期重置参考模型为当前策略
零梯度样本（DAPO 发现）	全组奖励相同导致无梯度信号	Dr.GRPO：过滤零梯度样本，token 级别计算损失

五、Qwen3.5 关键新增信息（2026.02）

Qwen3.5 于 2026 年 2 月 16 日发布，旗舰模型为 Qwen3.5-397B-A17B（397B 总参数，17B 激活参数）。

5.1 架构核心变化（相对 Qwen3）

（1）Gated Delta Networks（线性注意力）

- 采用 3:1 混合注意力布局：每 3 个 Transformer block 使用 Gated DeltaNet（线性注意力），每第 4 个 block 使用 Gated Attention（全注意力/softmax）
- Gated DeltaNet 结合了 Mamba2 的门控衰减机制与 Delta Rule 的隐状态更新方式
- 实现了相对序列长度的线性复杂度，使模型能够在极大上下文下保持性能

- 注意力输出门控机制有助于消除 Attention Sink 和 Massive Activation 问题，确保大规模训练数值稳定
- 相比 Qwen3 的纯 GQA 方案，这是一次根本性的注意力架构革新

（2）MoE 恢复共享专家

- Qwen3.5 重新引入共享专家机制：一个专用的 Dense MLP 处理所有 token 以捕获通用特征，提升训练稳定性
- 不同于 Qwen3 取消共享专家的策略，Qwen3.5 又加回来了

（3）原生多模态（Early Fusion）

- Qwen3.5 不再是独立的文本模型加视觉适配器，而是统一的视觉语言骨干，从预训练起就融合文本和多模态 token
- 视觉编码器采用 DeepStack ViT，使用 Conv3d 进行 patch embedding 以原生捕捉时间动态；融合多层特征而非仅用最后一层

（4）上下文与词表

- 原生支持 262,144 token 上下文，可通过 YARN 扩展至约 1M token
- 词表扩展到约 250K，对中文字符、数学符号和代码 token 编码更紧凑

（5）多语言

- 支持 201 种语言和方言（Qwen3 为 119 种）

5.2 训练流程

- 在万亿级多模态 token 上进行 Early Fusion 训练，多模态训练效率接近纯文本训练的 100%
- RL 训练扩展到百万级 Agent 环境，任务分布逐步增加复杂度，采用异步 RL 框架
- 解码吞吐量在 32K 上下文下比 Qwen3-Max 快 8.6 倍，256K 上下文下快 19 倍

5.3 模型规格一览

模型	类型	总参数	激活参数	上下文
Qwen3.5-397B-A17B	MoE	397B	17B	262K
Qwen3.5-122B-A10B	MoE	122B	10B	262K
Qwen3.5-35B-A3B	MoE	35B	3B	262K
Qwen3.5-27B	Dense	27B	27B	262K
Qwen3.5-9B	Dense	9B	9B	262K
Qwen3.5-4B	Dense	4B	4B	262K
Qwen3.5-0.8B	Dense	0.8B	0.8B	262K

六、各模型架构对比速查

模型	注意力	FFN / MoE	位置编码	训练精度	上下文
DeepSeek V3 (671B)	MLA	细粒度 MoE，256 专家选 8，1 共享	RoPE	FP8 (E4M3)	128K
DeepSeek R1 (671B)	MLA	同 V3	RoPE	FP8	128K
DeepSeek V3.2 (685B)	MLA + DSA	同 V3	RoPE	FP8	128K
Qwen3-MoE (235B-A22B)	GQA + QK-Norm	128 专家选 8，无共享	RoPE + YARN + DCA	BF16	128K
		MoE + 1 共享专家		BF16	262K（→1M）

Qwen3.5 (397B-A17B)	GQA + Gated DeltaNet (3:1)		RoPE + YARN		
Llama 3.1 (405B)	GQA	Dense FFN	RoPE	BF16	128K
Llama 4 Scout	GQA + iRoPE	MoE（16 专家选 1，交替层）	iRoPE	FP8	10M
Llama 4 Maverick	GQA + iRoPE	MoE（128 专家，交替层）	iRoPE	FP8	1M

七、FP8 训练专题详解（高频面试考点）

7.1 为什么要用 FP8？

传统 LLM 训练使用 BF16-FP32 混合精度。FP8 是从 16-bit 到 8-bit 的自然演进：

收益维度	具体效果
计算速度	H100/H200 Tensor Core 的 FP8 FLOPS 是 BF16 的 **2 倍**，实测端到端加速 **50~75%**
显存节省	权重/激活/梯度存储减半，单卡可训练更大模型（同集群下可训练约 2 倍参数的模型）
通信带宽	分布式训练中梯度通信量减半，减少跨节点通信开销
成本	DeepSeek V3 全量训练仅 278.8 万 H800 GPU 小时（约 557 万美元），FP8 是关键因素之一

7.2 FP8 的两种格式

格式	指数位 / 尾数位	最大值	用途	特点
E4M3	4位指数 / 3位尾数	±448	前向传播（权重、激活）	精度更高（尾数多），适合需要精细数值的前向计算
E5M2	5位指数 / 2位尾数	±57344	反向传播（梯度）	动态范围更大（指数多），适合梯度的大范围波动

混合策略：前向用 E4M3（精度优先），反向用 E5M2（范围优先）。这是 NVIDIA Transformer Engine 的默认配置。

> 💡 DeepSeek V3 的创新：前向和反向均使用 E4M3，通过细粒度量化（block-wise scaling）来弥补 E4M3 范围不足的问题，反而获得更好的精度。

7.3 FP8 训练的核心挑战

（1）动态范围窄 → 溢出/下溢

- E4M3 最大值仅 448，远小于 BF16 的 ~3.4×10^{38}
- 激活值中的离群值（outlier）可能远超 FP8 可表示范围，导致溢出
- 梯度值可能很小，低于 FP8 最小精度，导致下溢（梯度消失）

（2）量化误差累积

- 8-bit 精度低，每次运算引入的舍入误差更大
- 在数百层 Transformer 中逐层传播，可能导致训练不稳定甚至 loss 爆炸

（3）Attention Sink / Massive Activation

- Transformer 残差流中容易产生极端离群值（幅度可达标准差的 1000 倍以上）
- 这些离群值在 FP8 的窄范围内无法精确表示，是 FP8 训练不稳定的主因

7.4 解决方案全景

技术	说明	使用者
Per-tensor Scaling	每个张量维护一个缩放因子，将数值映射到 FP8 范围内	早期方案（粗粒度，易受离群值影响）
Block-wise / Tile-wise Scaling	将张量分成小块，每块一个独立缩放因子，离群值不会污染全局精度	**DeepSeek V3**（关键创新）、Ling 2.0
Delayed Scaling	基于历史 amax 统计来预估缩放因子，避免每步重新计算	NVIDIA Transformer Engine 默认方案
精度解耦	不同组件用不同精度：权重/激活用 FP8，master weights/optimizer states 保持 FP32	通用最佳实践
前向 E4M3 + 反向 E5M2	前向需要精度，反向需要范围，各取所需	标准混合方案（NVIDIA 推荐）
前向反向均 E4M3 + fine-grained scaling	DeepSeek 的创新：用细粒度缩放弥补 E4M3 范围不足，获得更好精度	DeepSeek V3/V3.2
TWEO 正则化	对残差块输出加正则项，抑制离群值产生，从源头解决问题	学术前沿（2025.11）

7.5 DeepSeek V3 的 FP8 实现要点（面试重点）

DeepSeek V3 是首个在超大规模（671B）模型上验证 FP8 训练可行性的工作：

- **量化粒度**：采用 block-wise scaling（细粒度量化），权重用 per-block 量化，激活用 per-token 量化
- **前向+反向均用 E4M3**：不用传统的 E4M3+E5M2 混合，而是全程 E4M3 + 细粒度缩放
- **算法-框架-硬件协同设计**：
  - 解决跨节点 MoE 训练的通信瓶颈
  - 实现计算与通信的近完全重叠
  - FP8 通信进一步减少分布式训练带宽需求
- **效果**：与 BF16 训练结果几乎无损，loss 相对误差约 0.001

7.6 哪些部分不能用 FP8?

- **Master weights、optimizer states**（一阶/二阶动量）通常保持 FP32
- **LayerNorm / RMSNorm** 的累加也常用高精度
- **嵌入层（Embedding）**：词表查找对精度敏感

八、面试一句话速记

考点	一句话答案
DeepSeek MLA 是什么?	将 KV 压缩为低秩隐向量，比 GQA 更激进地减少 KV cache，同时不损失甚至提升性能
DeepSeek DSA 是什么?	V3.2 新增的稀疏注意力，用 indexer 选 Top-k 个 token 做注意力，复杂度从 $O(L^2)$ 降到 $O(L \cdot k)$
GRPO 和 PPO 的区别?	GRPO 去掉 Critic 网络，从同一 prompt 的一组回答中计算相对优势（均值/标准差归一化），计算量减半
DeepSeek 无辅助损失路由?	取消辅助损失，改用 bias 项手动调节专家负载，负载均衡略差但模型质量更好
Qwen3 Thinking 模式?	单模型通过 chat template 切换思考/直答模式，支持 thinking budget 控制推理时思考量

Qwen3.5 Gated DeltaNet?	3:1 混合布局（3层线性注意力 + 1层全注意力），线性复杂度，支持超长上下文，消除 Attention Sink
Qwen3.5 相比 Qwen3 变了什么?	Gated DeltaNet 替代纯GQA；恢复共享专家；原生多模态 Early Fusion；上下文 262K；201种语言；250K词表
Llama 4 iRoPE?	交错使用 RoPE 层和无位置编码层，突破传统位置编码限制，支持最长 10M token
Llama 4 vs Llama 3 最大变化?	从 Dense 变为 MoE；从后融合变为 Early Fusion 多模态；上下文从 128K 到 10M；BF16 变 FP8
为什么用 FP8 训练?	H100 FP8 算力是 BF16 的 2 倍，内存减半，通信减半，DeepSeek V3 因此仅花费约 557 万美元完成训练
FP8 最大挑战?	动态范围窄（E4M3 最大仅 448），Transformer 中的激活离群值容易溢出导致训练崩溃
怎么解决离群值?	核心是 Scaling 策略：从 per-tensor → block-wise fine-grained scaling，局部缩放避免离群值污染全局
DeepSeek V3 FP8 有什么创新?	①前向反向均用 E4M3 + block-wise scaling（不用 E5M2）；②首次在 671B MoE 上验证可行；③FP8 通信减少分布式开销
哪些部分不能用 FP8?	Master weights、optimizer states（一阶/二阶动量）通常保持 FP32，LayerNorm/RMSNorm 累加也常用高精度
哪些模型用了 MTP?	DeepSeek V3（预测2个额外token，顺序预测保持因果链）；Qwen3.5 也支持 MTP speculative decoding
MLA vs GQA 核心区别	GQA 减少 KV 头数，MLA 对 KV 做低秩压缩，缓存低维潜变量，显存更低
MoE 推理瓶颈	All-to-All 通信：token 跨节点路由到不同专家，通信量随专家并行度线性增长
YaRN 外推原理	对 RoPE 频率重缩放：低频保持原始，高频压缩，插值而非外推
Attention Sink 成因	Transformer 需要"无操作"槽，首个 token 承接无意义权重，是量化离群值根源
DeepSeek-R1 涌现推理	GRPO + 可验证奖励 + 格式奖励，让模型自发学会长链推理，无需人工标注 CoT
共享专家作用	所有 token 必经的 Dense MLP，捕获通用特征，减少路由专家冗余，提升训练稳定性